



第15章： 模型量化、蒸馏与高效部署



学习目标与主线

学习目标与主线

01

理解为什么低比特能降低显存与加速推理，并能根据精度要求选择PTQ或QAT以及合适的量化粒度与混合精度策略。

02

理解教师—学生学习的核心机制，并能用合适的蒸馏目标与数据构建方法把大模型能力迁移到更小模型以满足部署约束。

03

理解从模型导出到推理引擎的关键环节，并能通过图优化、算子融合、批处理与KV缓存管理把量化/蒸馏的理论收益转化为真实的延迟与吞吐提升。



模型量化

量化基础：从浮点到低比特



LYCHEE



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

定义

量化 (Quantization) 是把模型推理时使用的数值表示从**高精度浮点** (FP16/BF16/FP32) 转换为**更低比特** 的表示 (INT8、INT4等)

常见做法：先量化权重，必要时再量化激活或KV缓存。

更快更省

低比特 会显著减少模型权重与部分中间数据的存储体积，从而降低显存/内存占用，并减少推理时的数据搬运量。

在很多硬件上，INT8/INT4有专门优化的矩阵乘实现，因此不仅“更省带宽”，也可能“计算更快”。

最终表现为延迟下降、吞吐上升、同样硬件可服务更多请求。

模型量化

精度下降

低比特表示是把连续浮点映射到更少的离散刻度，刻度变少会引入舍入与截断误差，这些误差会在网络层间传播并累积，导致输出分布发生偏移。

对比维度	PTQ 训练后量化	QAT 量化感知训练
定义	训练完成后再把权重/激活从浮点转换为低比特，不再训练或只做少量校准	在训练/微调阶段把量化误差加入前向，让模型在训练中适应低比特噪声
什么时候做	模型训练完之后、部署前	训练阶段或部署前的再训练/微调阶段
是否需要训练	通常不需要重新训练，最多需要少量校准数据	需要训练或微调（有训练循环与梯度更新）
主要优点	成本低、速度快、落地最简单，工程上最常作为第一步	精度更稳，尤其更容易在INT4等更低比特下保持效果
主要缺点	对敏感层/异常值多的模型可能掉精度明显，低比特（如INT4）更容易不稳	训练成本高、流程更复杂、需要数据与训练资源，调参工作量更大
更适合的目标比特	INT8较常见且更稳；INT4可行但更依赖模型与校准质量	更适合冲击INT4及更激进设置，同时保持精度
典型使用方式	“先试水”：快速看收益与精度损失，作为默认起点	“补救/加强”：PTQ不够时，用QAT把精度拉回来并稳定部署

量化对象全景：权重、激活与KV缓存

如何用更低比特表示权重和中间数据来显著降低显存/内存与带宽开销

权重量化

把模型参数权重从FP16/BF16等浮点压到INT8/INT4，通常带来最直接的收益，显著减少模型体积与显存占用，并且推理引擎支持成熟，所以加速效果相对稳定、最适合作为量化的第一步。

激活量化

把推理时各层产生的中间激活也转换为低比特，进一步降低中间张量的读写与内存压力，但由于激活分布会随输入内容和长度变化，动态范围更不稳定，因此更容易引入误差，落地难度和调参成本通常高于权重量化。

KV量化

把注意力机制中的Key/Value缓存用低比特存储，专门针对长上下文场景的显存瓶颈，能明显降低KV占用、提高并发与吞吐，不过需要重点评估对生成质量、上下文一致性和长程依赖能力的影响。

经验

通常只做权重量化就能获得明显且稳定的收益，而更激进、资源更受限或长上下文压力更大的场景才会进一步考虑激活量化与KV缓存量化。

低比特量化的难点：异常值与敏感层

现象与结论

低比特量化（尤其INT4）掉精度往往集中在少数“敏感层”，而不是全层平均变差；因此能否成功通常取决于这几层的处理策略。换句话说，找准并保护关键层，比盲目把所有层都压到同一比特更重要

典型后果

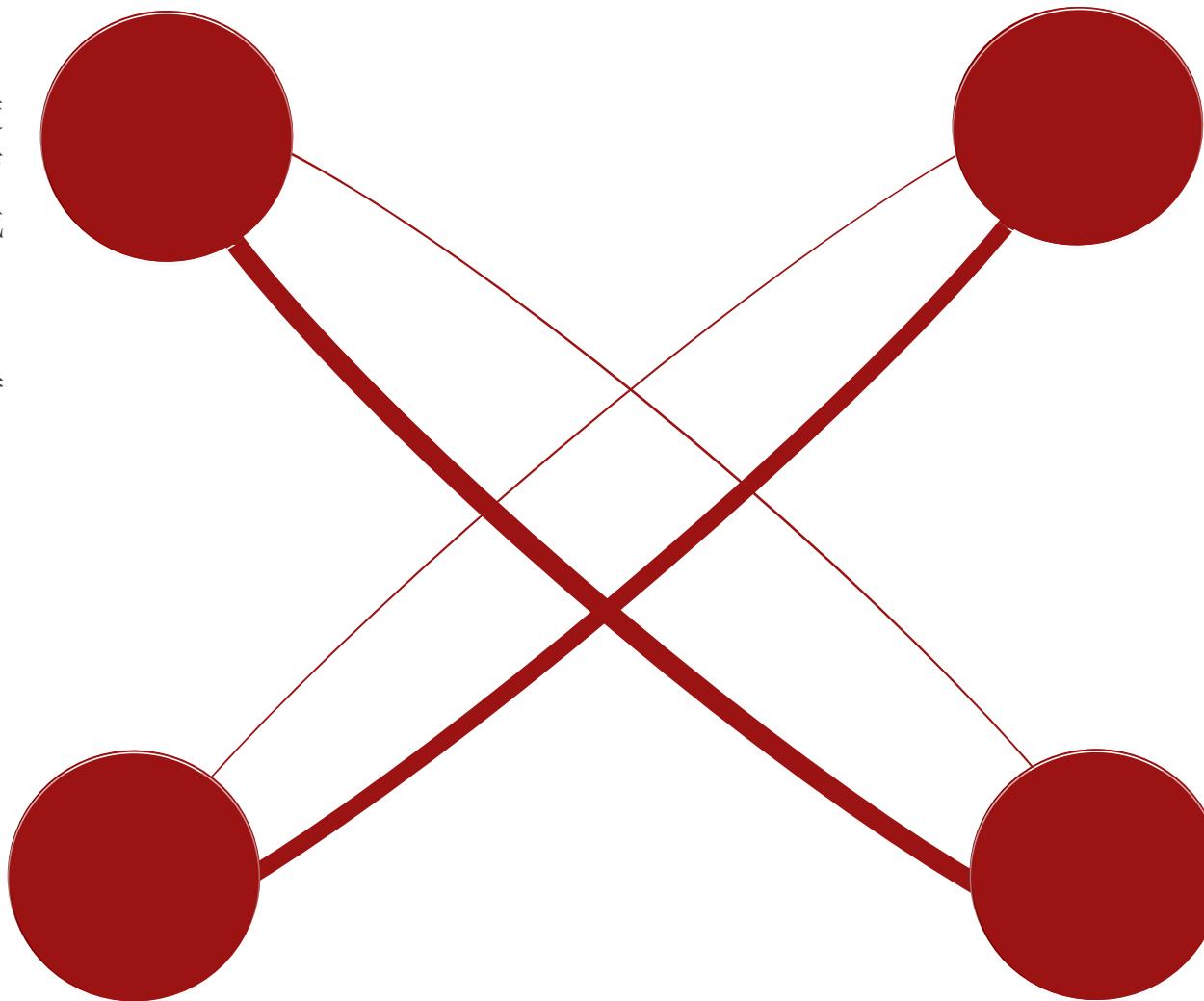
这些误差会在层与层之间传播，并在自回归生成中逐步累积，表现为输出概率分布异常（过尖或过平）、困惑度上升、生成质量下降。更典型的是在长文本、推理题或少数“困难样本”上突然崩坏，出现明显不稳定。

为什么敏感

敏感层常见于注意力相关投影、输出层以及归一化附近层，它们更容易出现**异常大值（outlier）**或通道尺度差异很大；一旦动态范围被少数大值拉宽，量化刻度就被迫变粗，导致大部分正常值被“压缩”到更少的离散点上。结果就是信息分辨率下降，误差在这些层上被放大。

工程做法

最常用的是混合精度量化，即大多数层用INT8/INT4拿收益，但对敏感层保留FP16/BF16以稳住质量；同时可以对异常通道单独保高精度或采用更细粒度（按通道/按组）缩放来减少outlier拖累。若PTQ仍不够稳，再用QAT在微调中让模型适应量化噪声，通常能显著改善INT4下的稳定性。



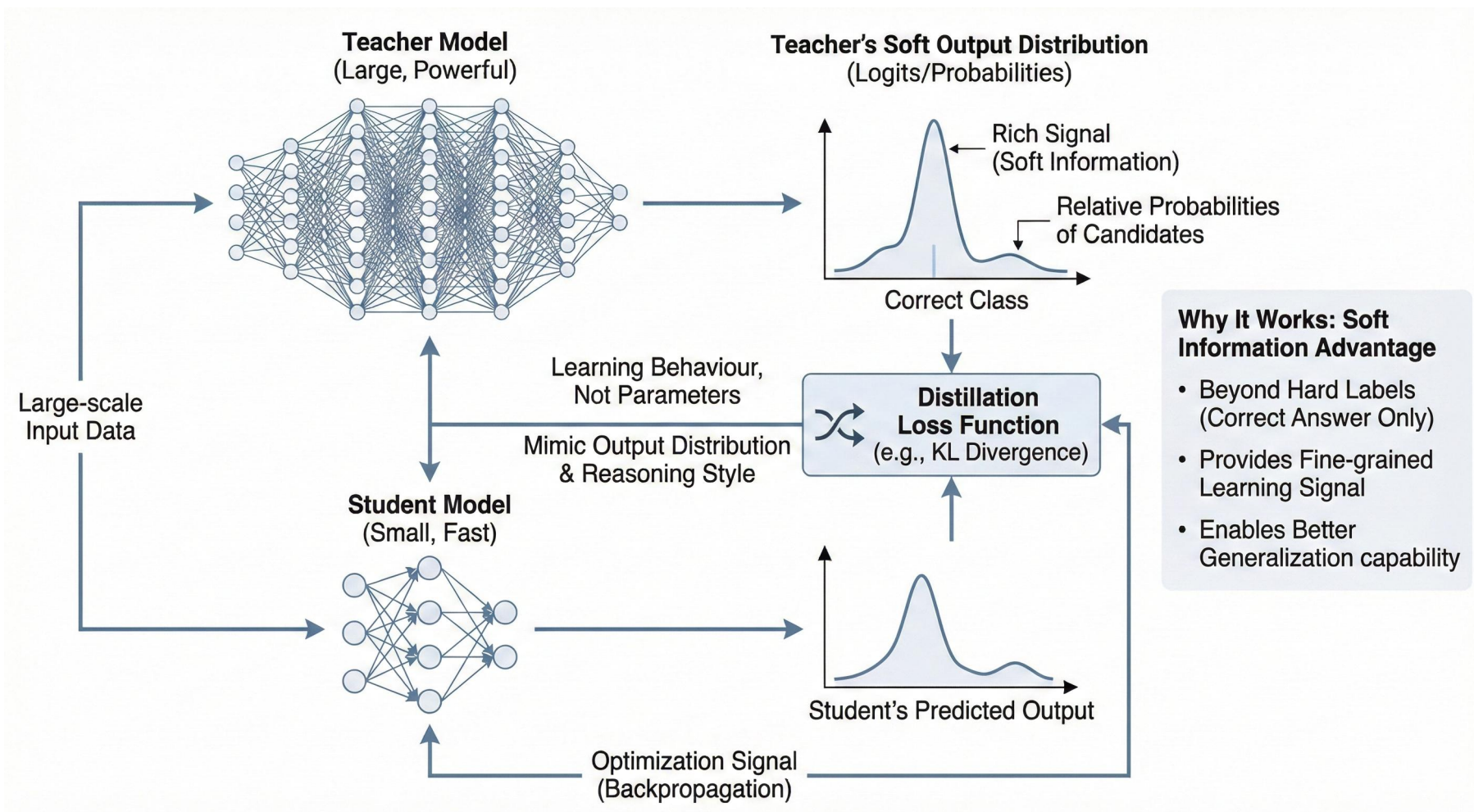


模型蒸馏

蒸馏基础：把大模型的“能力”教给小模型

核心逻辑

让学生模型在大量输入上模仿教师的输出分布、回答偏好与推理风格，从而用更小的模型获得接近教师模型的效果。



蒸馏之所以有效，是因为教师模型输出除了正确答案，还包含各候选之间的**相对概率与置信度结构**。

这种更细的学习信号比只用硬标签更丰富，能帮助学生模型学到**更平滑、更可泛化的决策边界**，通常在小模型、数据有限或任务复杂时更明显。

三大目标：logits、特征、任务

logits

➤ logits蒸馏

让学生去匹配教师的输出概率分布，也就是学习教师在每个候选类别或每个词上的“概率分配方式”，不仅学会选对答案，还学会教师的置信度结构，因此常用于分类任务与语言建模等场景。

特征

➤ 特征蒸馏

让学生的中间层表示尽量接近教师的中间层表示，相当于让学生在内部表征上复现教师的特征抽取过程，这类方法通常更适合教师与学生结构相似、层之间能够对齐的情况。

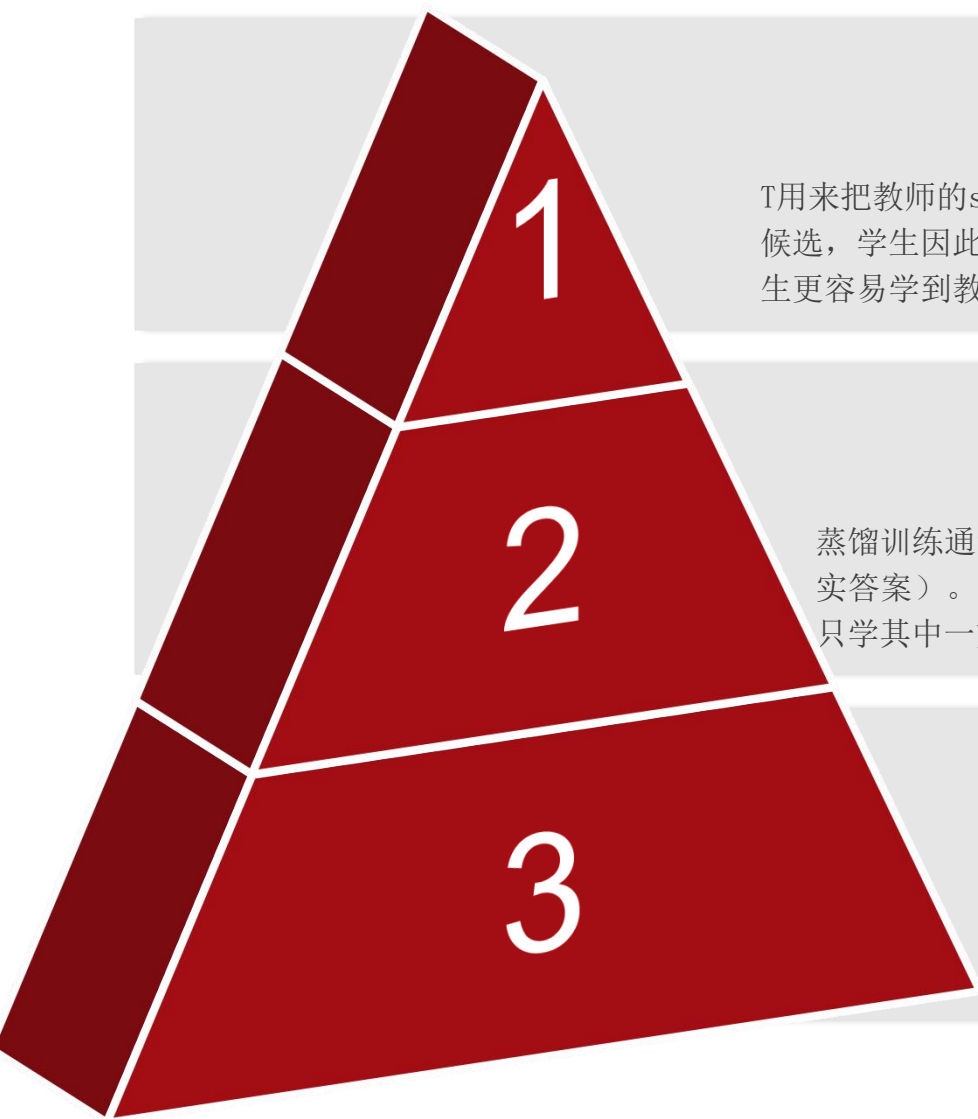
任务

➤ 任务蒸馏

把教师当作“数据生成器/标注器”，由教师生成高质量的训练数据，例如指令跟随数据、问答对、对话轨迹、偏好对比数据，甚至解释链与思考过程，然后用这些数据去训练学生完成 SFT（监督微调）或偏好对齐，从而把教师的能力迁移到更小模型上

目标

蒸馏训练要素：温度、损失与权重平衡



1. 温度 (Temperature, T) ▶▶▶

T 用来把教师的softmax输出“变软”，让概率不只集中在最高概率词/类别上，而是把更多信息分配给次优候选，学生因此能学到更丰富的相对偏好与置信度结构。它重要在于能**显著提升蒸馏信号的信息量**，让学生更容易学到教师的细腻行为，而不是只学一个“对/错”的结果。

2. 损失 (Loss) ▶▶▶

蒸馏训练通常同时包含蒸馏损失（让学生匹配教师的输出分布）和监督损失（让学生匹配真实标签/真实答案）。它重要在于把“像教师一样输出”和“对齐真实正确答案”同时纳入优化目标，避免学生只学其中一方而产生偏差。

3. 权重 (Weights) ▶▶▶

需要给蒸馏损失与监督损失设置权重，用来控制学生更偏向“模仿教师”还是更偏向“遵循真实标签”。它重要在于权重决定了最终模型的行为取舍：只学教师会继承教师偏差，只学硬标签又可能学不到教师的概率偏好与生成风格，因此必须通过权重在不同任务上找到平衡点。

量化 vs 蒸馏：两条模型压缩路线如何选

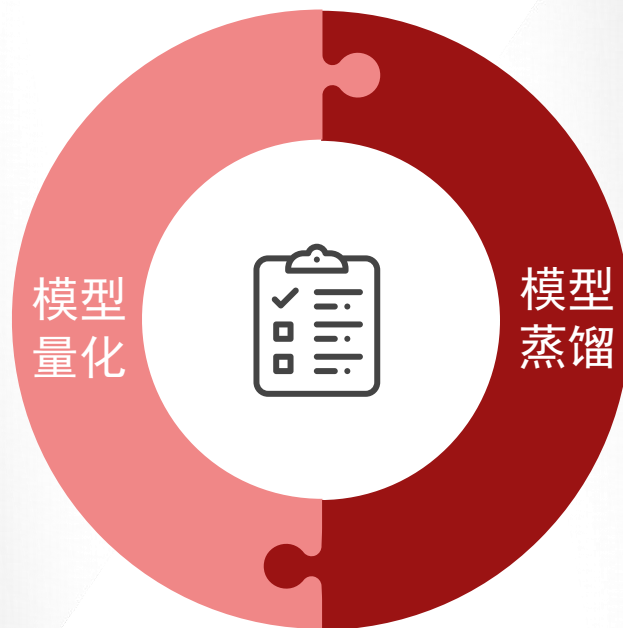
快速降本加速（量化）与从根本变小（蒸馏）的取舍与组合策略

量化在不改变模型结构的前提下，把权重或中间数据从FP16/BF16等浮点换成INT8/INT4等低比特，从而显著降低显存/内存占用与带宽开销，并在很多硬件上获得更快的矩阵乘推理速度。

最大的特点是**落地快、训练成本低**（PTQ往往只需少量校准数据），非常适合“已有可用模型，想快速降本加速”的场景；但低比特越激进越容易遇到敏感层与异常值导致的**精度下降**，需要混合精度、合适的粒度/校准，必要时用QAT来稳住效果。

落地快

成本低



蒸馏通过教师模型把输出分布、偏好与推理风格迁移给学生模型，直接把模型做小，**从根本上降低参数规模与推理计算量**，更适合端侧部署、低成本推理或需要更小包体的场景。它的特点是需要数据与训练/微调流程（成本高于纯PTQ），但一旦蒸馏成功，小模型在特定任务或领域上往往能接近大模型表现；在追求更低比特仍保持稳定时，蒸馏也常与QAT和部署优化组合使用，以同时兼顾“模型更小”和“低比特更稳”。

表现优

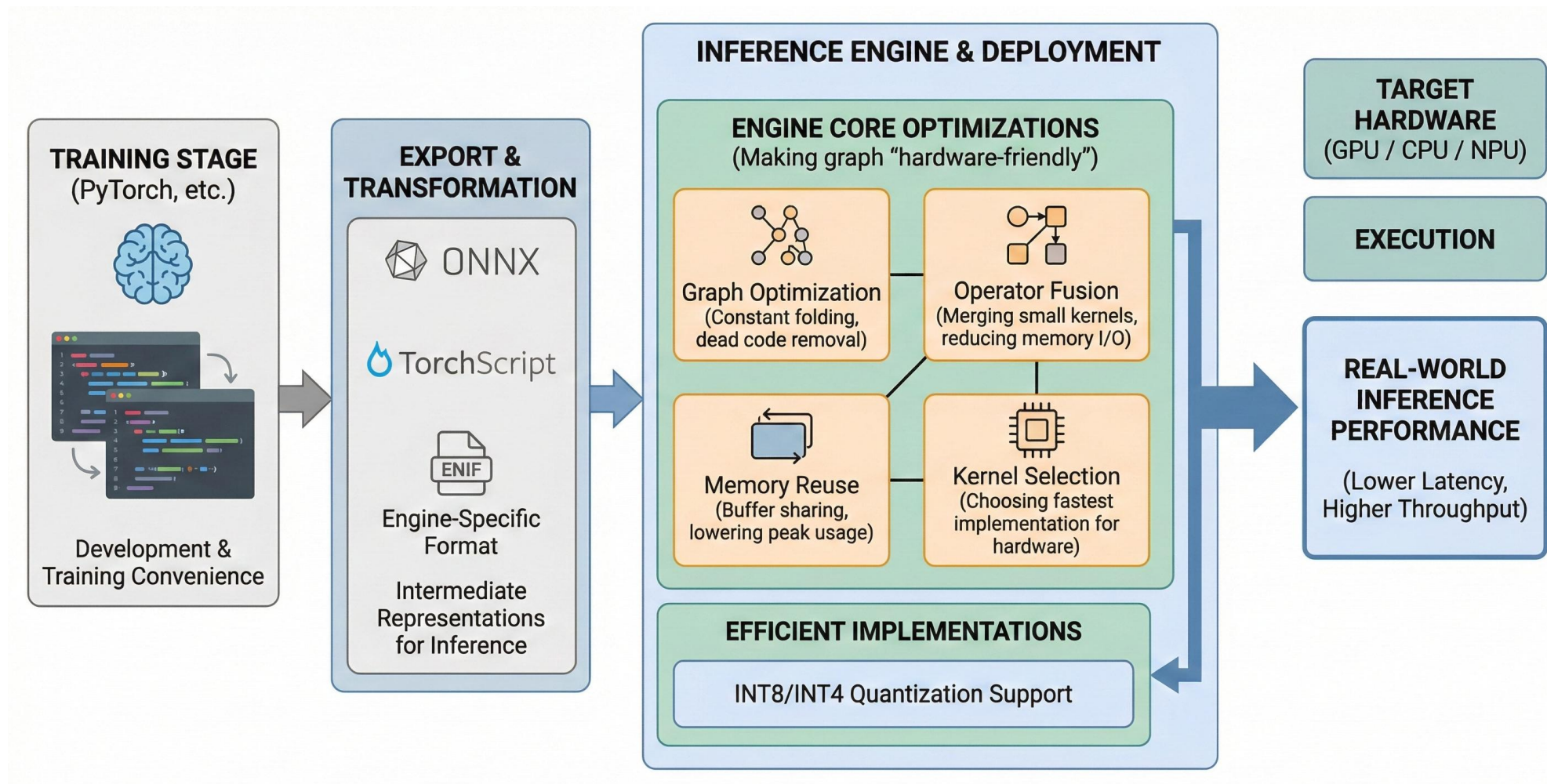
更稳定



模型高效部署

高效部署全流程：从 PyTorch 到推理引擎落地

模型通常先在 PyTorch 等训练框架中完成训练与验证，部署时再将其导出为更适合推理的表示形式（如 ONNX、TorchScript 或推理引擎专用格式），最后由推理引擎在目标硬件（GPU/CPU/NPU）上高效执行。



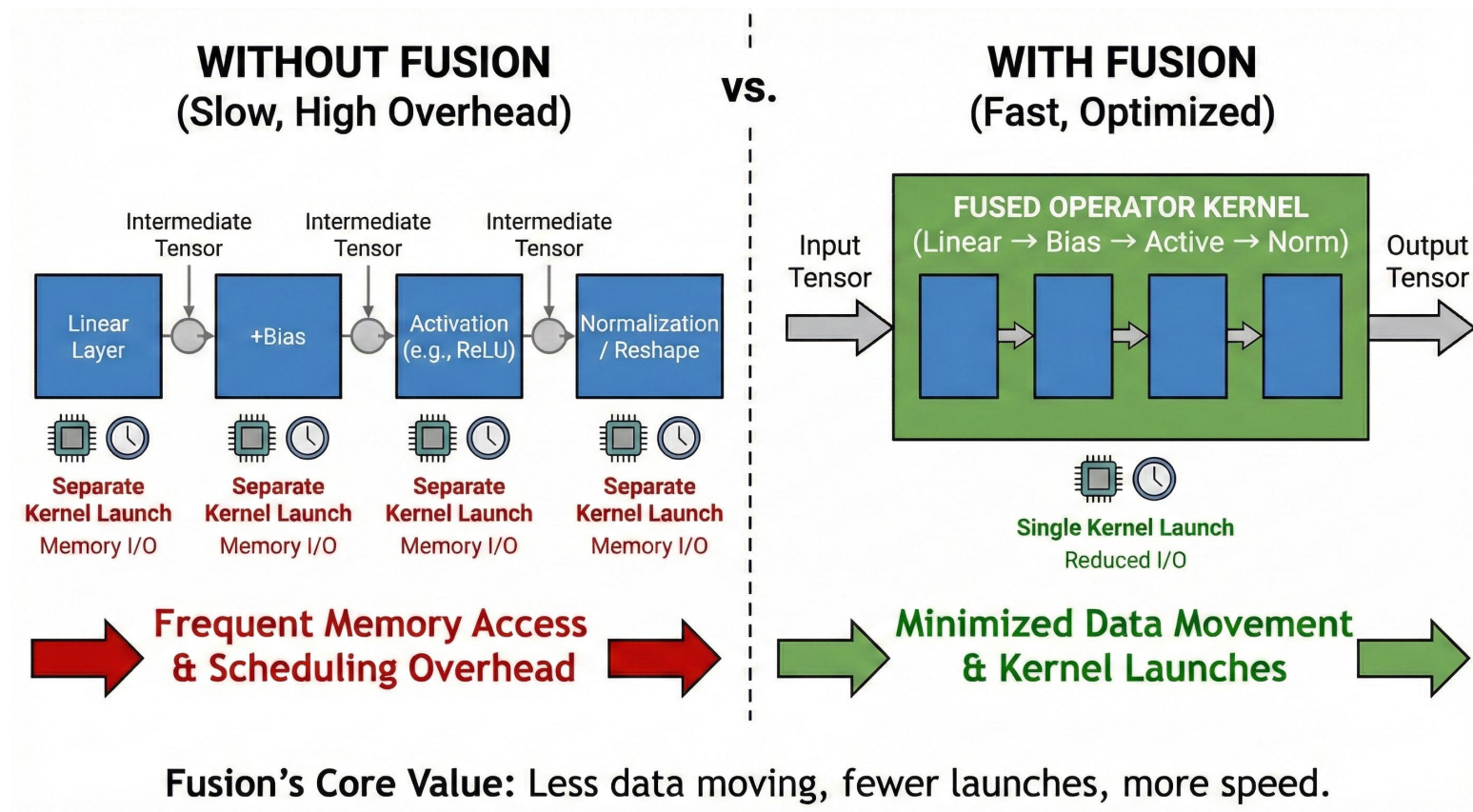
编译与图优化：算子融合为何能加速

核心原理

尽量少搬数据、少启动 kernel。

在 Transformer 中，常见的计算链是“线性层 → 加 bias → 激活 → 归一化 / reshape”等多个小算子串联；如果逐个算子执行，每一步都会产生中间张量并触发多次内存读写与 kernel 启动，带来明显的带宽与调度开销。

算子融合 (Fusion) 把这串连续操作合并到一次 kernel 中完成，减少中间结果落地、降低内存访问次数，同时减少启动与调度开销，因此推理速度更快。直觉上，现代硬件很多时候不是“算不动”，而是“被内存读写和调度开销拖慢”。

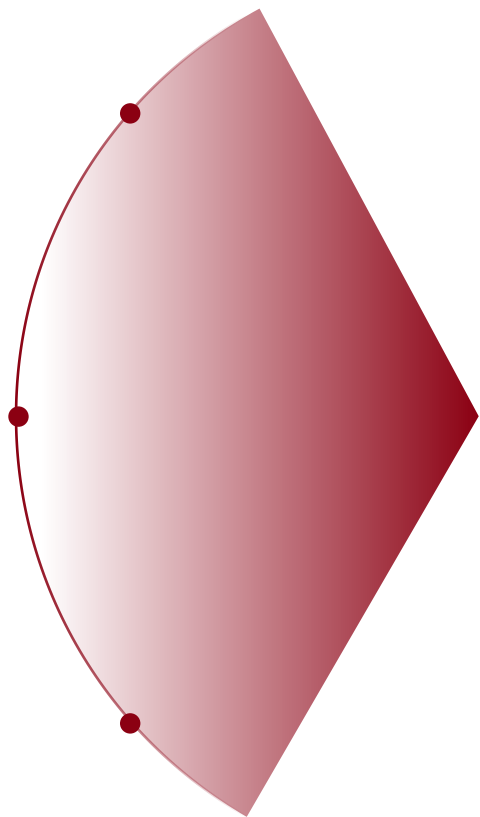


并行与批处理：吞吐怎么做上去

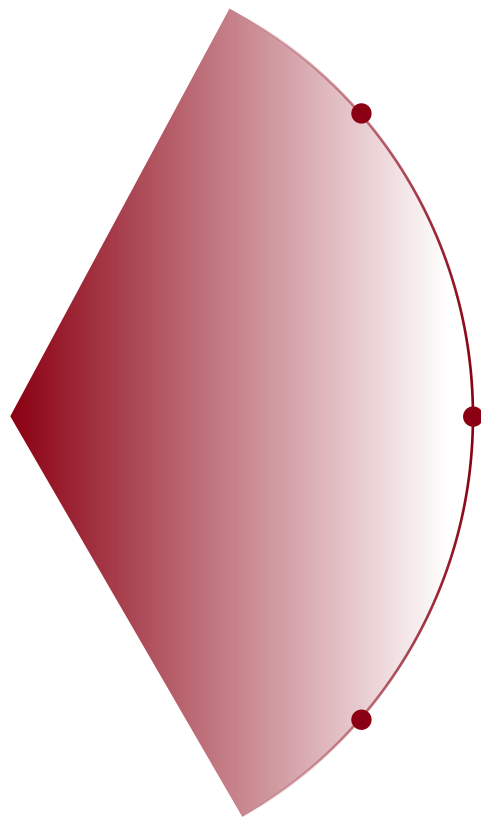
批处理 (Batching) 的核心是把多个请求合并计算，提高矩阵乘等核心算子的利用率，让硬件并行度更高，从而显著提升吞吐，但通常会带来一定的单请求延迟增加。

流水线与异步的核心是把数据准备、CPU/GPU拷贝与GPU计算尽量重叠起来，**减少等待时间**，让设备更少“空转”，提升整体处理效率。

批处理



多卡并行



多卡并行（如张量并行、流水线并行）可以把更大的模型装进多张卡并提升总体吞吐，但代价是引入跨卡通信与同步开销，配置不当反而会变慢。

对入门者最重要的经验是先把单卡单机的链路跑顺并用 profiling 找清瓶颈，再逐步扩展到并行方案，否则很容易出现“越优化越慢”的结果。

01

KV 缓存

是注意力机制在生成时保存的 Key/Value 历史状态，用来避免每生成一个新 token 都重复计算之前的注意力信息；因此它会随着生成步数增加、上下文变长而持续增长，占用大量显存/内存。

02

KV 占用一旦过大

可能就会压缩可用显存、限制并发请求数量，并且带来更重的读写带宽压力，从而直接降低吞吐、拉高成本，尤其在对话与长文本生成场景更明显。

优化策略

- 使用更高效的 KV 内存布局与分页/分块管理，减少内存碎片与频繁拷贝；
- 对 KV 做量化以显著降低缓存占用并提升并发能力；
- 在产品层面选择合适的上下文窗口与截断/摘要策略，在体验与成本之间做平衡。

很多对话应用的主要成本并不在模型权重本身，而在长上下文带来的 KV 缓存开销

端侧部署现实：CPU / GPU / NPU 的差异与取舍

侧端部署

把训练好的模型放到用户设备本地（如手机、平板、可穿戴、边缘网关等）的 CPU/GPU/NPU 上运行，实现不依赖云端或少依赖云端的实时推理与应用。

并行度高、擅长大规模矩阵乘与张量计算，适合跑深度学习里最重的线性层和注意力计算，因此在服务器侧通常能提供最高的吞吐与较低的单位推理成本。

优势往在于成熟的推理生态与丰富的优化路径，比如**高效的内核、图优化、混合精度与多卡并行**等。局限是功耗高、硬件成本高，并且在某些场景会被显存容量与带宽限制。

通用性强、部署门槛低、系统兼容性好，几乎所有设备都有 CPU，因此非常适合做“随处可跑”的推理落地。它的短板是单位算力相对弱，深度学习推理很容易被计算与内存带宽拖慢，所以通常更依赖 INT8 等低比特量化和高度优化的算子内核来提升速度。

CPU 在**小批量、低并发**、对延迟敏感但模型不太大的场景里比较常见，也常作为边缘设备的主力执行单元。

为神经网络推理专门设计，能在较低功耗下提供很高的推理效率，因此非常适合移动端与嵌入式端侧的“省电长时间运行”场景。

它通常对**量化支持更强**，尤其偏好 INT8/INT4 等低比特路径，但代价是对算子种类、算子组合和数据布局要求更严格，很多模型需要做算子替换或结构调整才能完全跑在 NPU 上。

GPU

CPU

NPU

最终 “能稳定运行、体验可接受、成本可控” 的整体工程效果。